

1. 생성 패턴(Creational Patterns)

① 싱글톤(Singleton) 패턴

```
// Singleton.java
public class Singleton {
    private static Singleton instance;

    private Singleton() {} // 외부 생성 금지

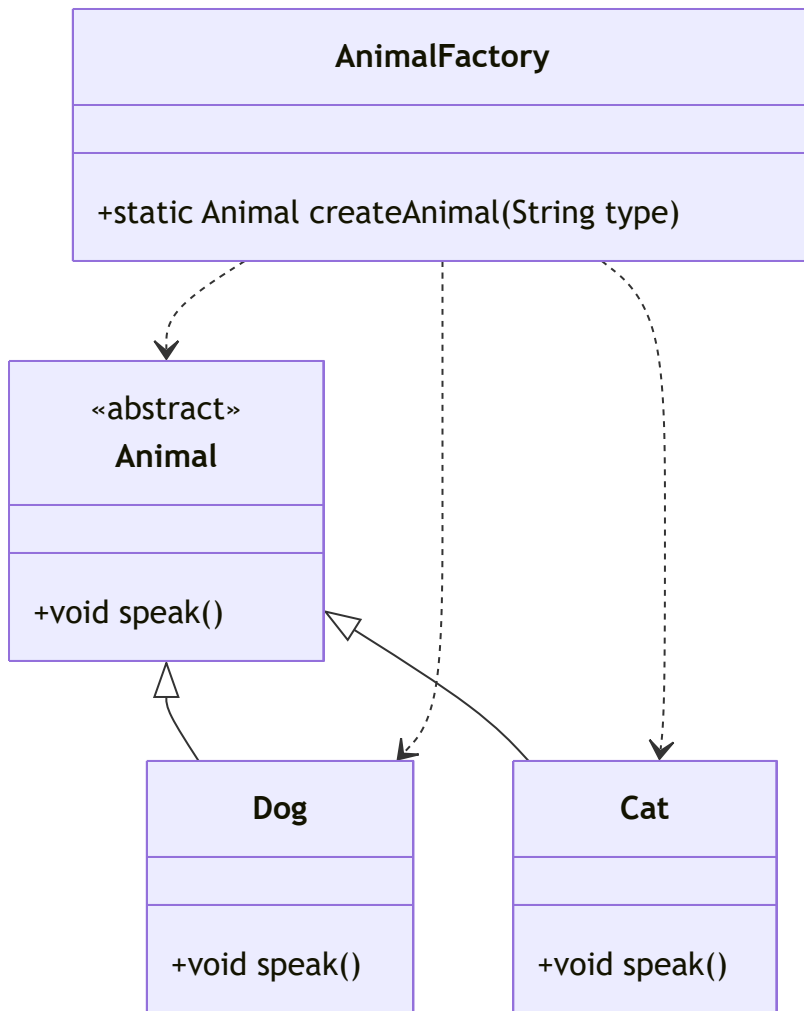
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("하나뿐인 인스턴스입니다!");
    }
}
```

Singleton
- static Singleton instance
- Singleton() + static Singleton getInstance() + void showMessage()

② Factory Method 패턴

```
abstract class Animal {  
    abstract void speak();  
}  
  
class Dog extends Animal {  
    void speak() { System.out.println("멍멍!"); }  
}  
  
class Cat extends Animal {  
    void speak() { System.out.println("야옹!"); }  
}  
  
class AnimalFactory {  
    public static Animal createAnimal(String type) {  
        return switch (type) {  
            case "dog" -> new Dog();  
            case "cat" -> new Cat();  
            default -> throw new IllegalArgumentException("Unknown type");  
        };  
    }  
}
```



③ Abstract Factory 패턴

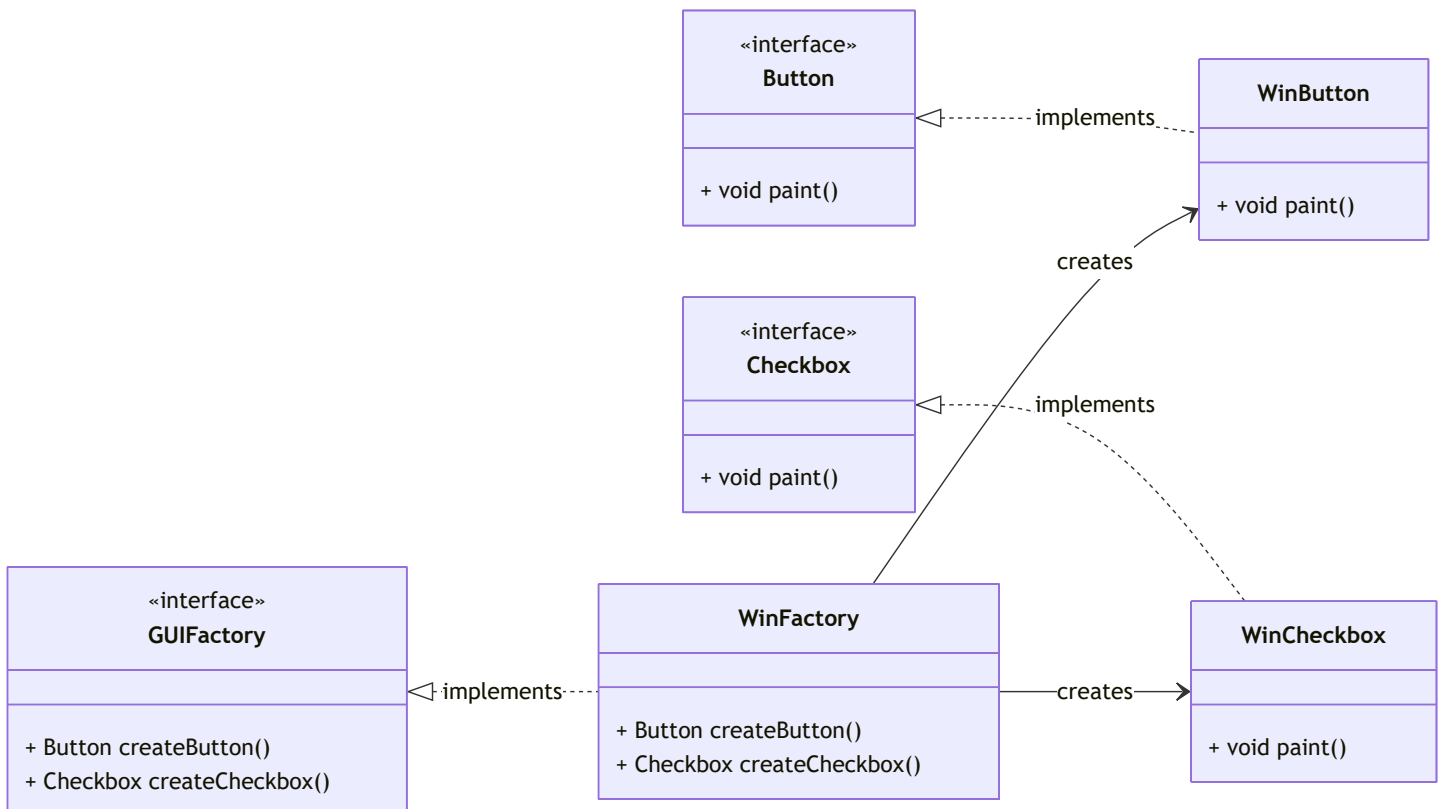
```
interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}
```

```
interface Button { void paint(); }  
interface Checkbox { void paint(); }
```

```
class WinFactory implements GUIFactory {  
    public Button createButton() { return new WinButton(); }  
    public Checkbox createCheckbox() { return new WinCheckbox(); }  
}
```

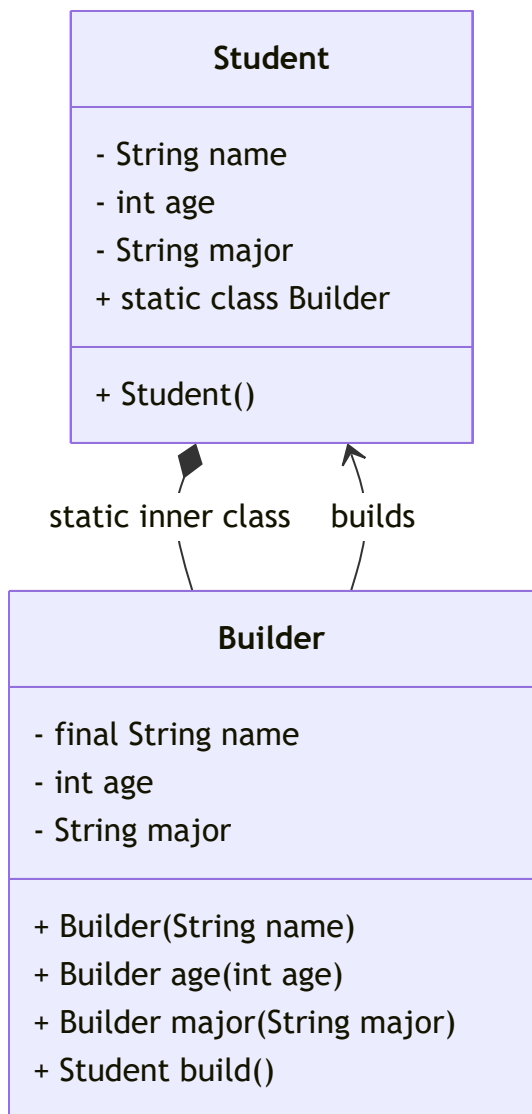
```
class WinButton implements Button {  
    public void paint() { System.out.println("Windows 버튼 생성"); }  
}
```

```
class WinCheckbox implements Checkbox {  
    public void paint() { System.out.println("Windows 체크박스 생성"); }  
}
```



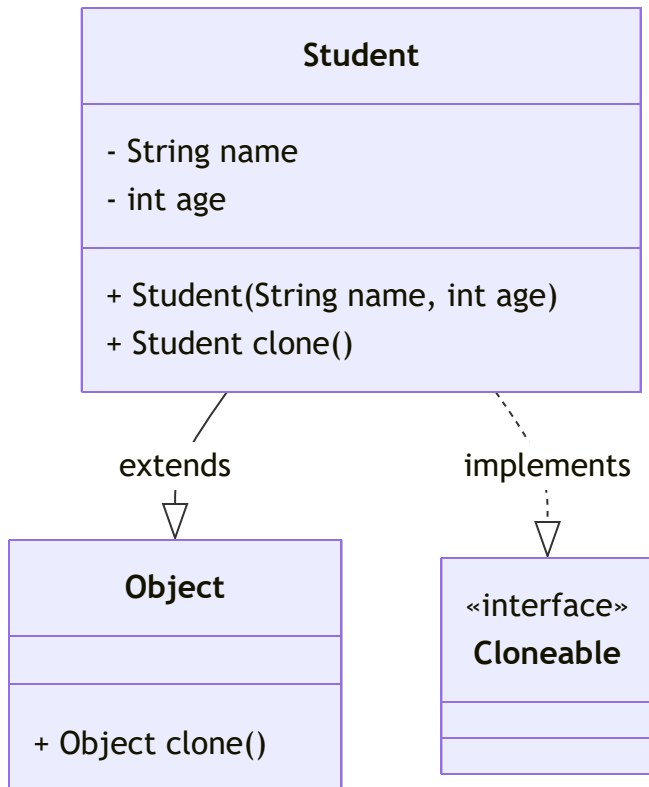
④ Builder 패턴

```
class Student {  
    private String name;  
    private int age;  
    private String major;  
  
    public static class Builder {  
        private final String name;  
        private int age;  
        private String major;  
  
        public Builder(String name) { this.name = name; }  
  
        public Builder age(int age) { this.age = age; return this; }  
        public Builder major(String major) { this.major = major; return this; }  
  
        public Student build() {  
            Student s = new Student();  
            s.name = name; s.age = age; s.major = major;  
            return s;  
        }  
    }  
}
```



⑤ Prototype 패턴

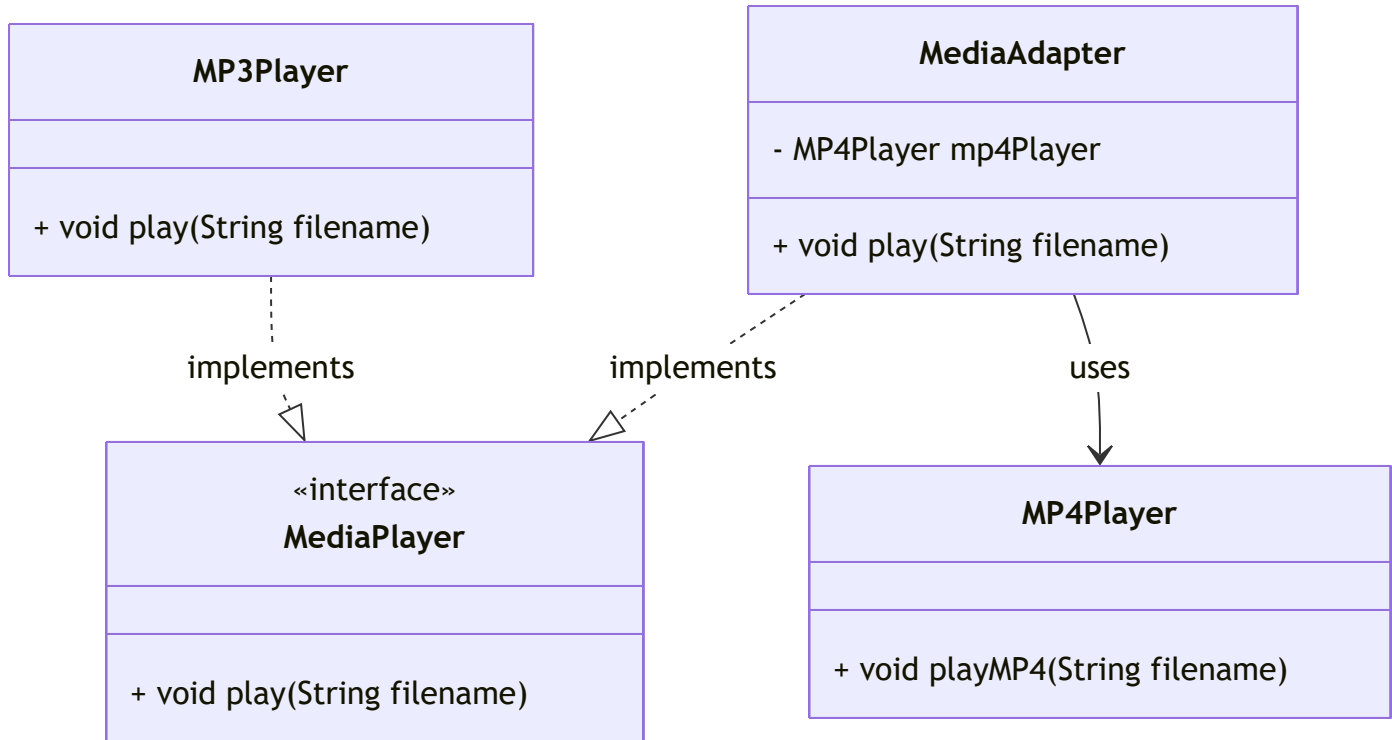
```
class Student implements Cloneable {  
    String name;  
    int age;  
  
    public Student(String name, int age) {  
        this.name = name; this.age = age;  
    }  
  
    public Student clone() {  
        try {  
            return (Student) super.clone();  
        } catch (CloneNotSupportedException e) {  
            throw new AssertionError();  
        }  
    }  
}
```



2. 구조 패턴 (Structural Patterns)

① Adapter 패턴

```
interface MediaPlayer {  
    void play(String filename);  
}  
  
class MP3Player implements MediaPlayer {  
    public void play(String filename) {  
        System.out.println("MP3 파일 재생: " + filename);  
    }  
}  
  
class MP4Player {  
    void playMP4(String filename) {  
        System.out.println("MP4 파일 재생: " + filename);  
    }  
}  
  
class MediaAdapter implements MediaPlayer {  
    private MP4Player mp4Player = new MP4Player();  
    public void play(String filename) {  
        mp4Player.playMP4(filename);  
    }  
}
```

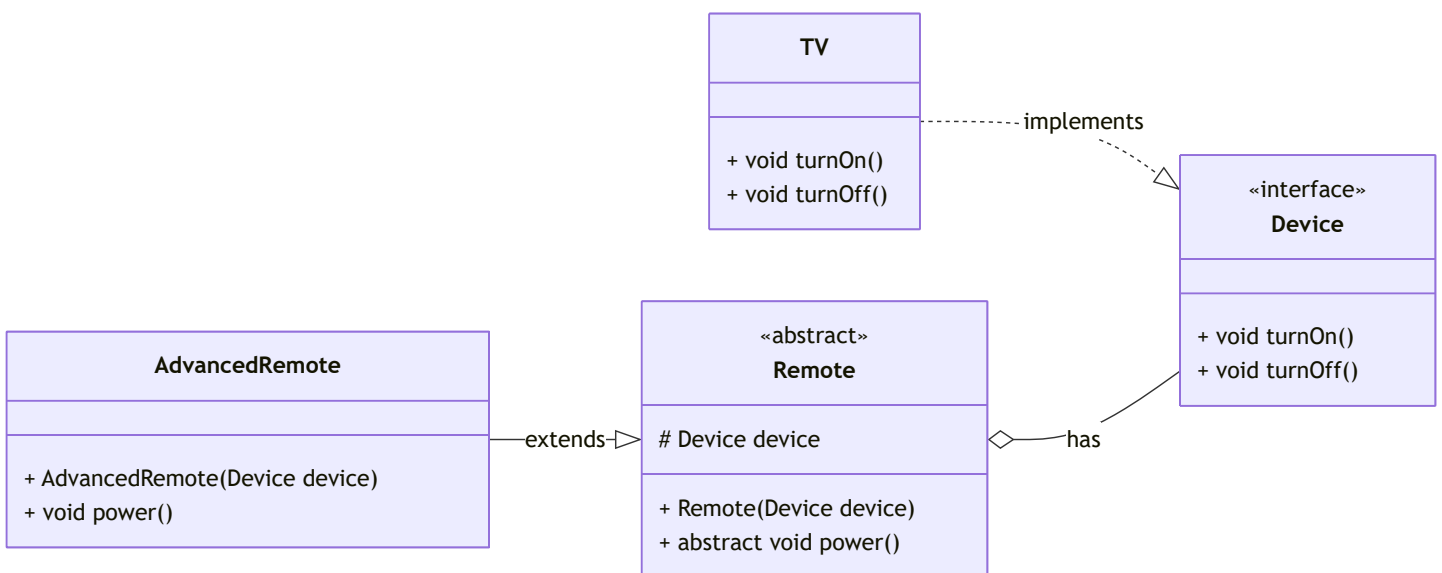
② Bridge 패턴

```
interface Device {  
    void turnOn();  
    void turnOff();  
}
```

```
class TV implements Device {  
    public void turnOn() { System.out.println("TV ON"); }  
    public void turnOff() { System.out.println("TV OFF"); }  
}
```

```
abstract class Remote {  
    protected Device device;  
    Remote(Device device) { this.device = device; }  
    abstract void power();  
}
```

```
class AdvancedRemote extends Remote {  
    AdvancedRemote(Device device) { super(device); }  
    void power() {  
        System.out.println("리모컨 전원 조작");  
        device.turnOn();  
    }  
}
```



③ Composite 패턴

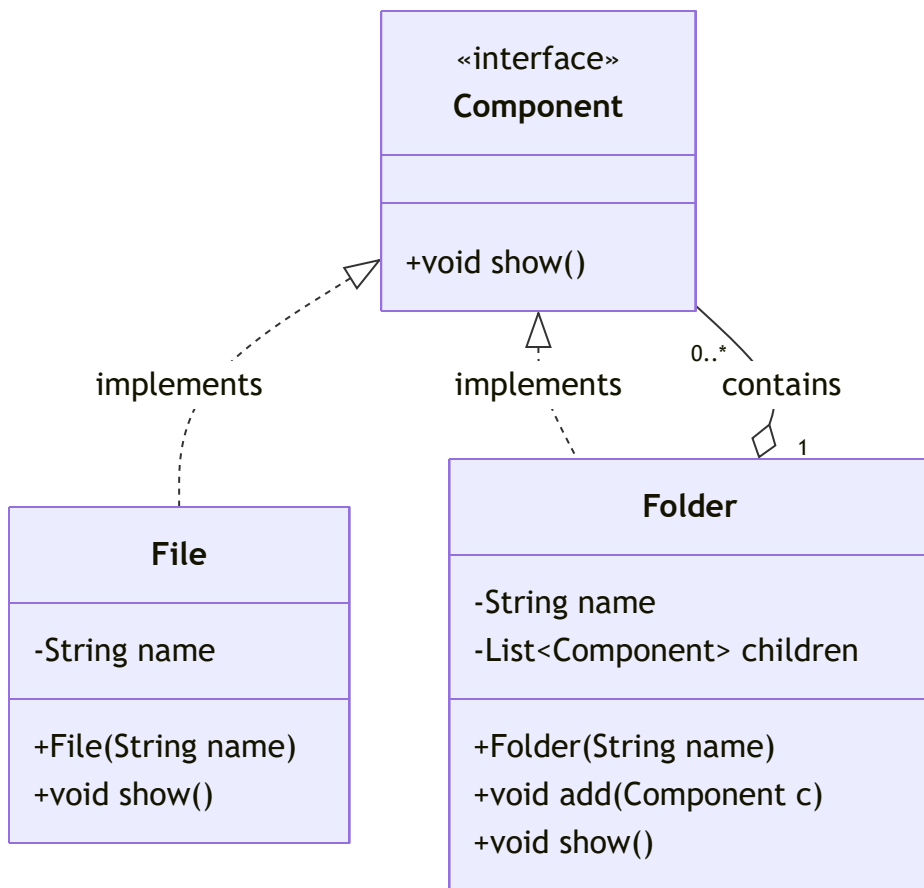
```
import java.util.*;

interface Component {
    void show();
}

class File implements Component {
    String name;
    File(String name) { this.name = name; }
    public void show() { System.out.println("- " + name); }
}

class Folder implements Component {
    String name;
    List<Component> children = new ArrayList<>();
    Folder(String name) { this.name = name; }

    public void add(Component c) { children.add(c); }
    public void show() {
        System.out.println("[폴더] " + name);
        for (Component c : children) c.show();
    }
}
```



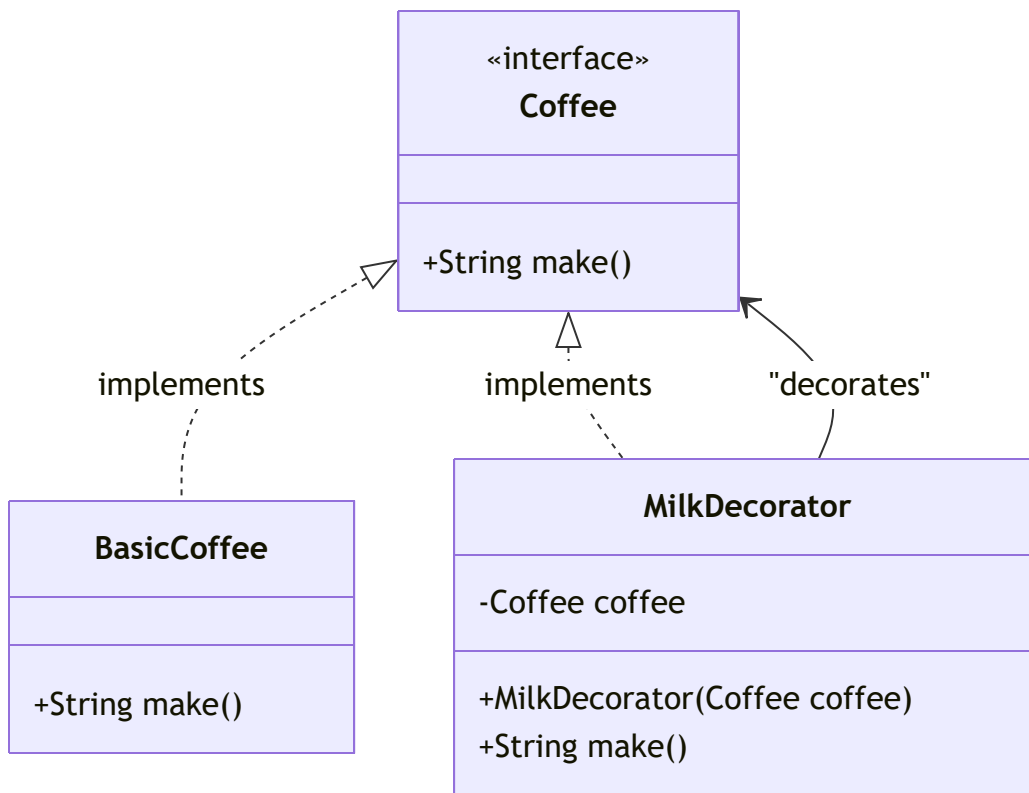
④ Decorator 패턴

```

interface Coffee {
    String make();
}

class BasicCoffee implements Coffee {
    public String make() { return "커피"; }
}

class MilkDecorator implements Coffee {
    private Coffee coffee;
    MilkDecorator(Coffee coffee) { this.coffee = coffee; }
    public String make() { return coffee.make() + " + 우유"; }
}
  
```



⑤ Facade 패턴

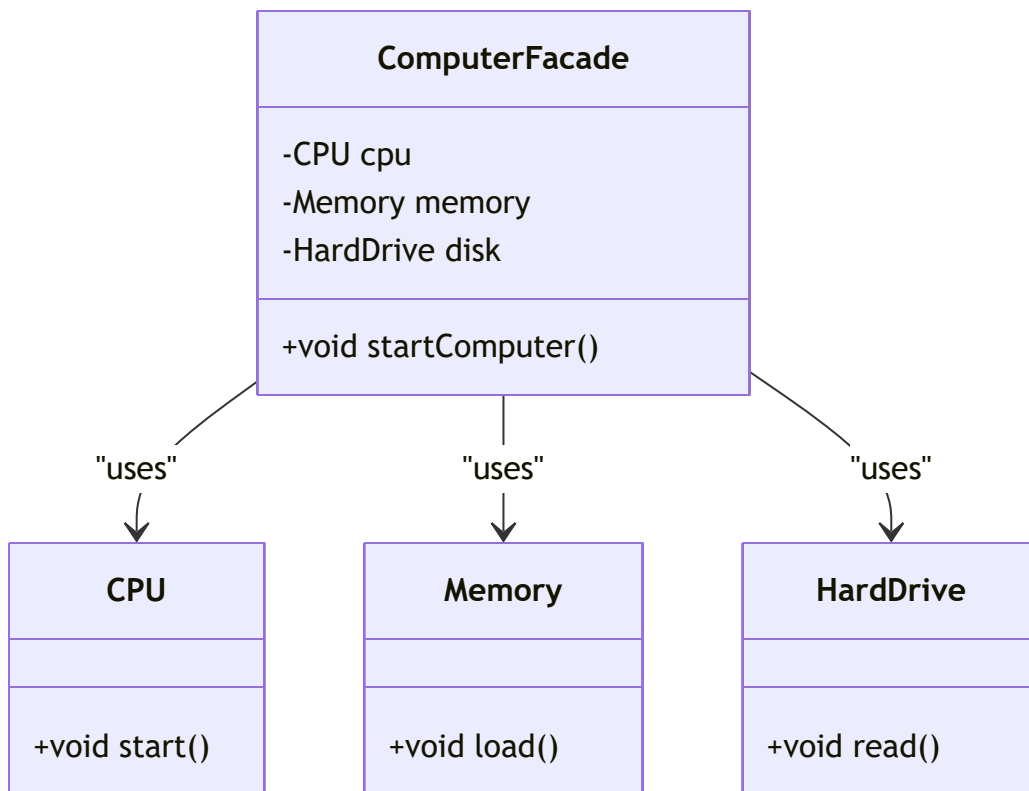
```

class CPU { void start() { System.out.println("CPU 시작"); } }
class Memory { void load() { System.out.println("메모리 로드"); } }
class HardDrive { void read() { System.out.println("디스크 읽기"); } }
  
```

```

class ComputerFacade {
    private CPU cpu = new CPU();
    private Memory memory = new Memory();
    private HardDrive disk = new HardDrive();

    void startComputer() {
        cpu.start();
        memory.load();
        disk.read();
        System.out.println("컴퓨터 부팅 완료");
    }
}
  
```

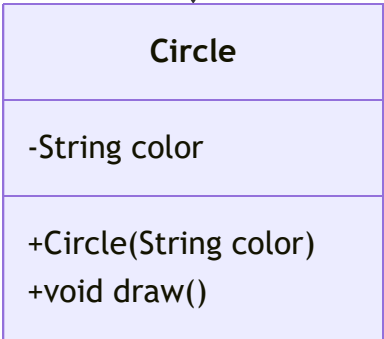
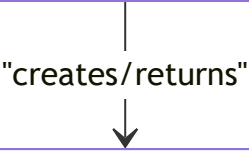
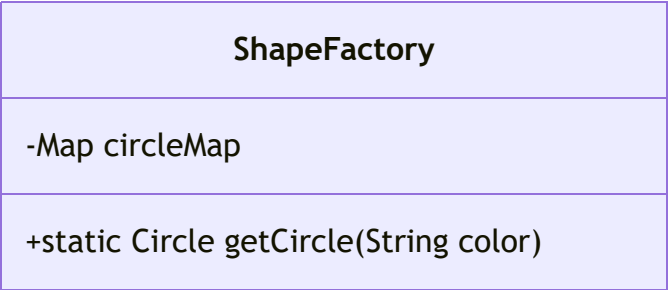


⑥ Flyweight 패턴

```
import java.util.*;

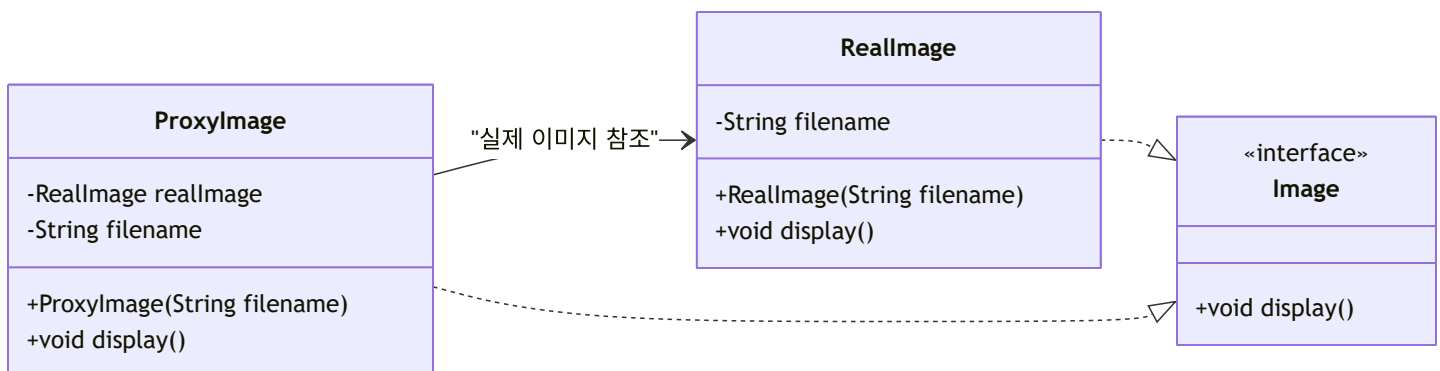
class ShapeFactory {
    private static final Map<String, Circle> circleMap = new HashMap<>();
    public static Circle getCircle(String color) {
        return circleMap.computeIfAbsent(color, Circle::new);
    }
}

class Circle {
    private String color;
    Circle(String color) { this.color = color; }
    void draw() { System.out.println(color + " 원 그리기"); }
}
```



⑦ Proxy 패턴

```
interface Image {  
    void display();  
}  
  
class RealImage implements Image {  
    private String filename;  
    RealImage(String filename) { this.filename = filename; }  
    public void display() { System.out.println(filename + " 표시 중"); }  
}  
  
class ProxyImage implements Image {  
    private RealImage realImage;  
    private String filename;  
  
    ProxyImage(String filename) { this.filename = filename; }  
  
    public void display() {  
        if (realImage == null) {  
            realImage = new RealImage(filename);  
        }  
        System.out.println("Proxy를 통해 이미지 접근");  
        realImage.display();  
    }  
}
```



3. 행위(Behavioral) 패턴

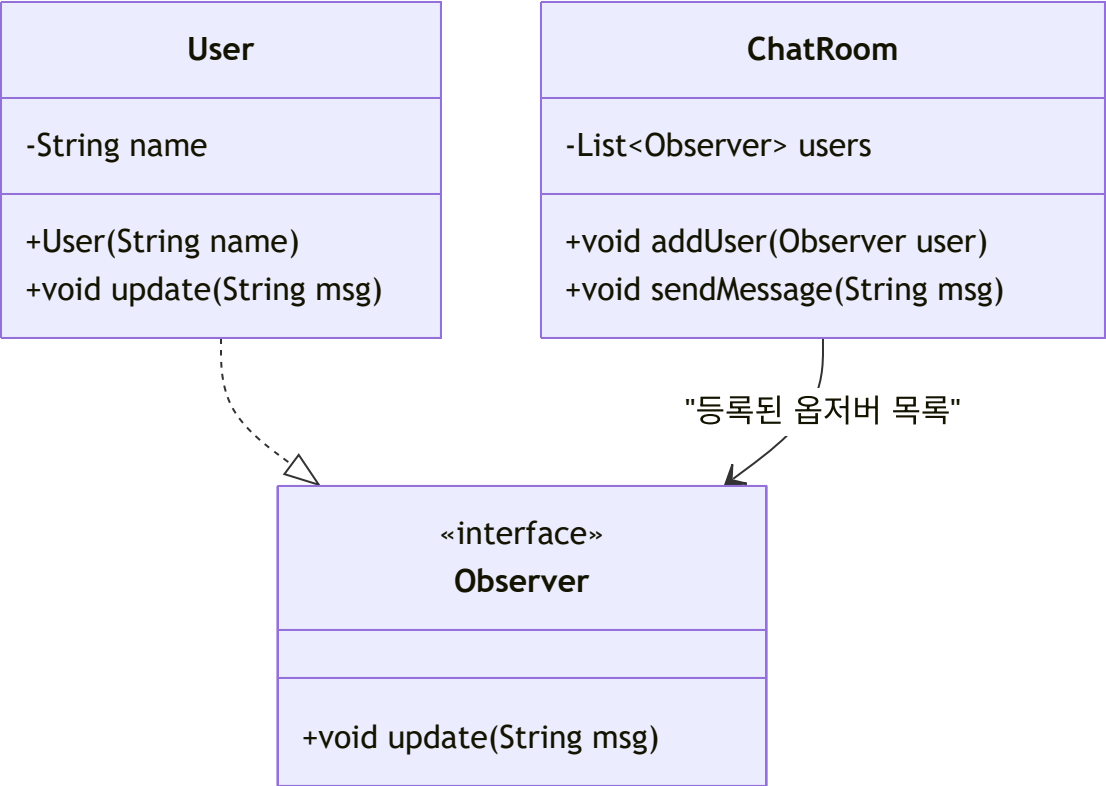
① Observer 패턴

```
import java.util.*;

interface Observer {
    void update(String msg);
}

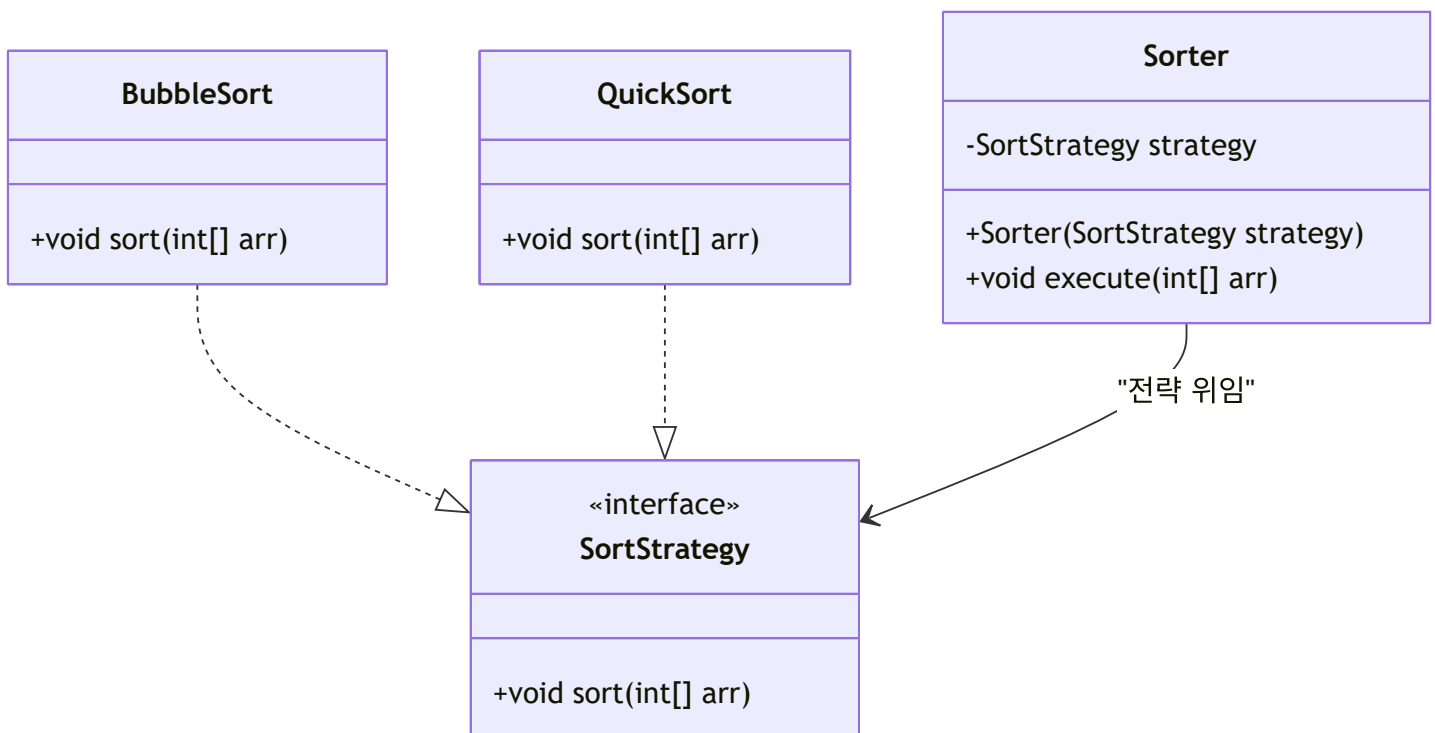
class User implements Observer {
    private String name;
    User(String name) { this.name = name; }
    public void update(String msg) {
        System.out.println(name + " 알림: " + msg);
    }
}

class ChatRoom {
    private List<Observer> users = new ArrayList<>();
    void addUser(Observer user) { users.add(user); }
    void sendMessage(String msg) {
        for (Observer u : users) u.update(msg);
    }
}
```



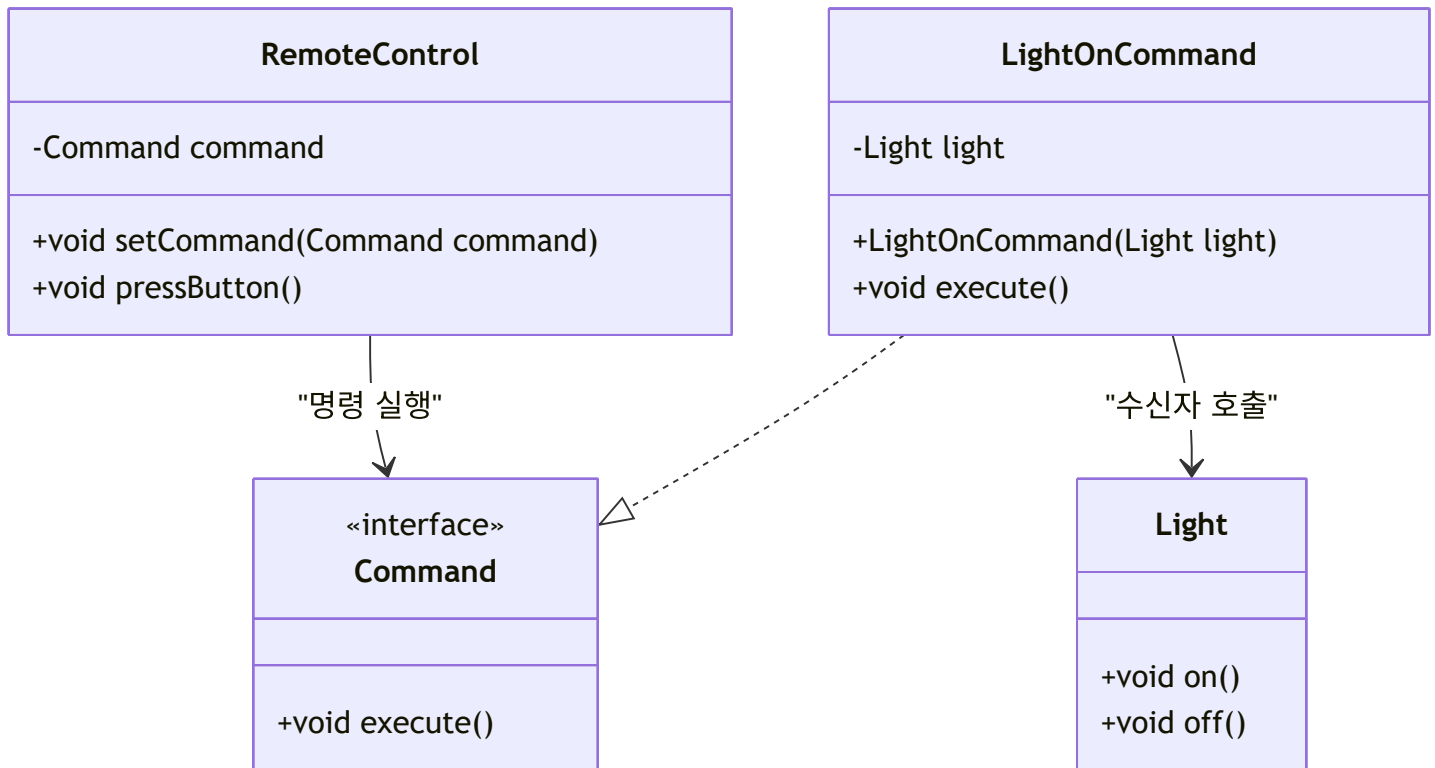
② Strategy 패턴

```
interface SortStrategy {  
    void sort(int[] arr);  
}  
  
class BubbleSort implements SortStrategy {  
    public void sort(int[] arr) {  
        System.out.println("버블 정렬 수행");  
    }  
}  
  
class QuickSort implements SortStrategy {  
    public void sort(int[] arr) {  
        System.out.println("퀵 정렬 수행");  
    }  
}  
  
class Sorter {  
    private SortStrategy strategy;  
    Sorter(SortStrategy strategy) { this.strategy = strategy; }  
    void execute(int[] arr) { strategy.sort(arr); }  
}
```



③ Command 패턴

```
interface Command {  
    void execute();  
}  
  
class Light {  
    void on() { System.out.println("조명 ON"); }  
    void off() { System.out.println("조명 OFF"); }  
}  
  
class LightOnCommand implements Command {  
    private Light light;  
    LightOnCommand(Light light) { this.light = light; }  
    public void execute() { light.on(); }  
}  
  
class RemoteControl {  
    private Command command;  
    void setCommand(Command command) { this.command = command; }  
    void pressButton() { command.execute(); }  
}
```



④ Template Method 패턴

```

abstract class Game {
    abstract void initialize();
    abstract void play();
    abstract void end();

```

```

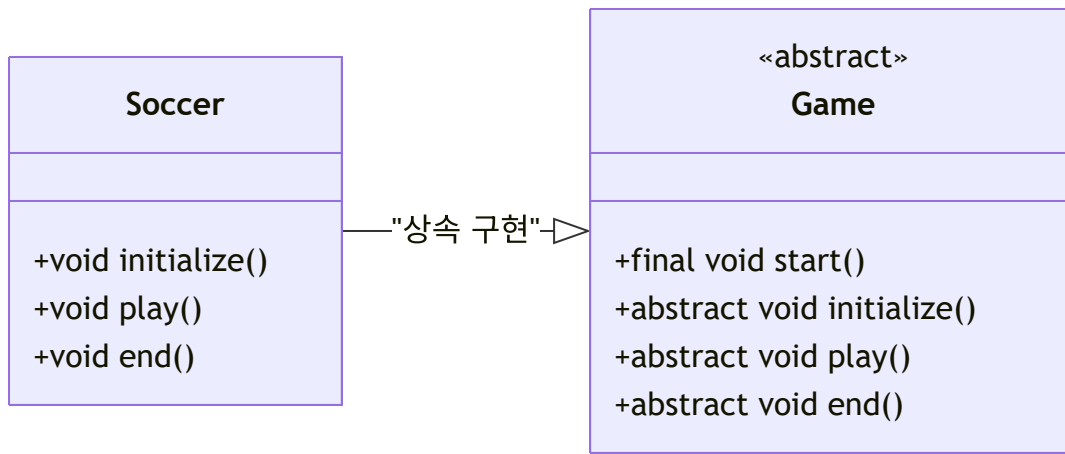
    public final void start() {
        initialize();
        play();
        end();
    }
}

```

```

class Soccer extends Game {
    void initialize() { System.out.println("축구 시작"); }
    void play() { System.out.println("경기 중..."); }
    void end() { System.out.println("경기 종료"); }
}

```



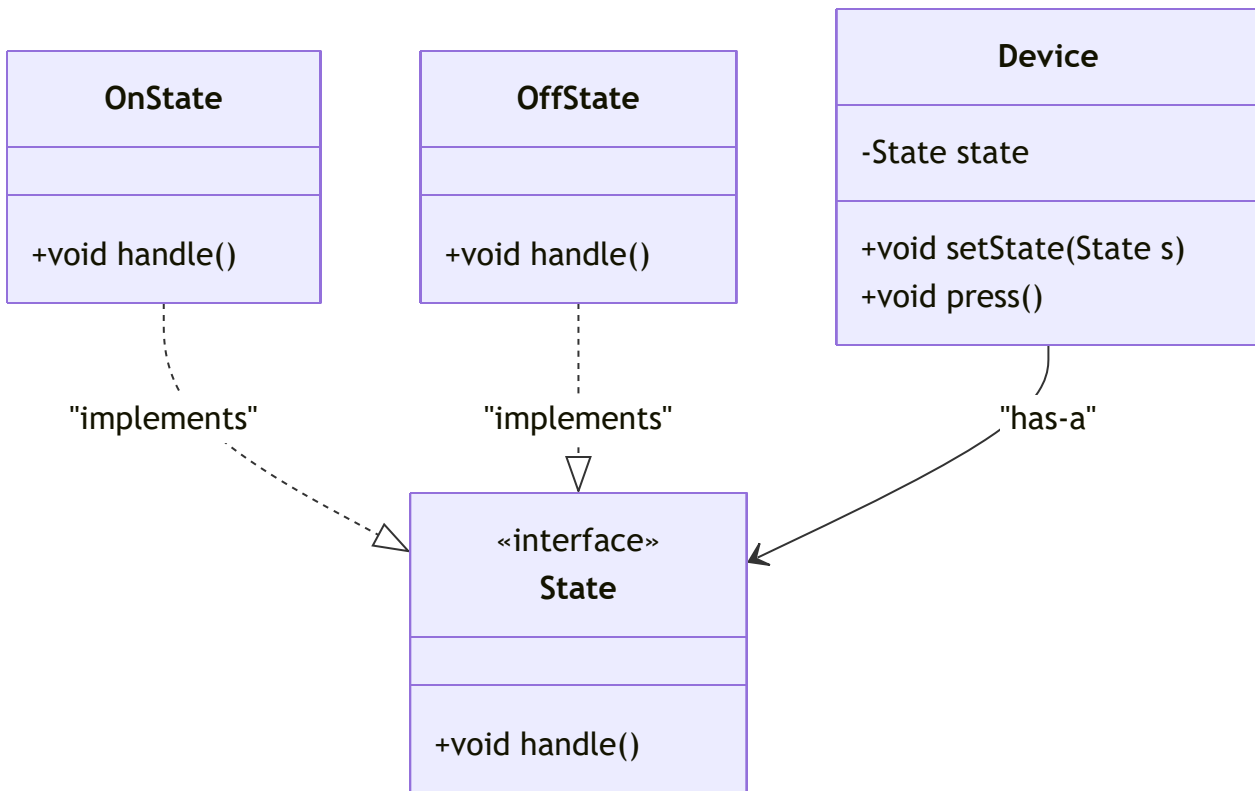
⑤ State 패턴

```
interface State {
    void handle();
}

class OnState implements State {
    public void handle() { System.out.println("기기가 켜져 있습니다."); }
}

class OffState implements State {
    public void handle() { System.out.println("기기가 꺼져 있습니다."); }
}

class Device {
    private State state;
    void setState(State s) { this.state = s; }
    void press() { state.handle(); }
}
```

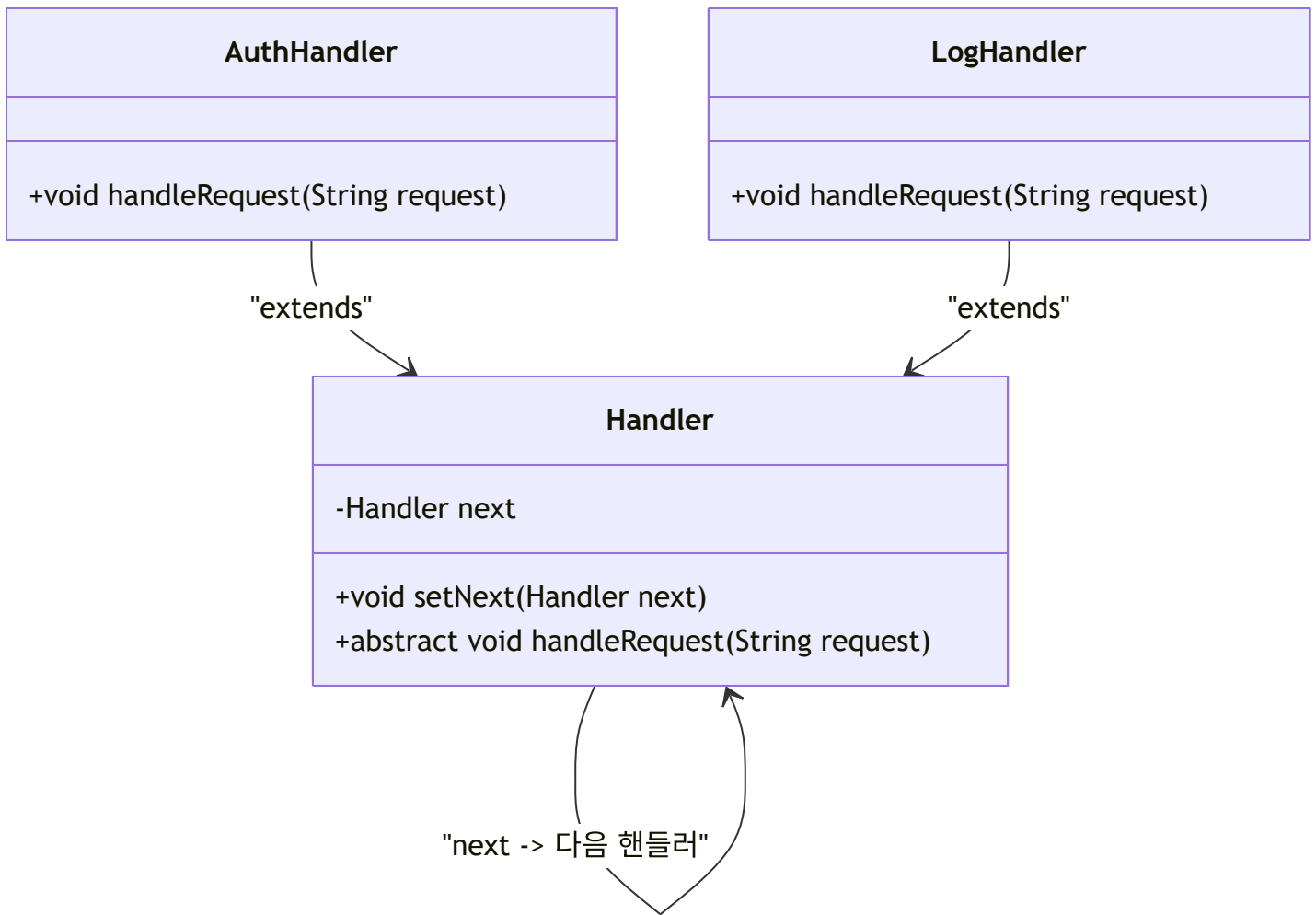


⑥ Chain of Responsibility 패턴

```
abstract class Handler {
    protected Handler next;
    void setNext(Handler next) { this.next = next; }
    abstract void handleRequest(String request);
}

class AuthHandler extends Handler {
    void handleRequest(String request) {
        if (request.equals("auth")) System.out.println("인증 완료");
        else if (next != null) next.handleRequest(request);
    }
}

class LogHandler extends Handler {
    void handleRequest(String request) {
        if (request.equals("log")) System.out.println("로그 기록");
        else if (next != null) next.handleRequest(request);
    }
}
```

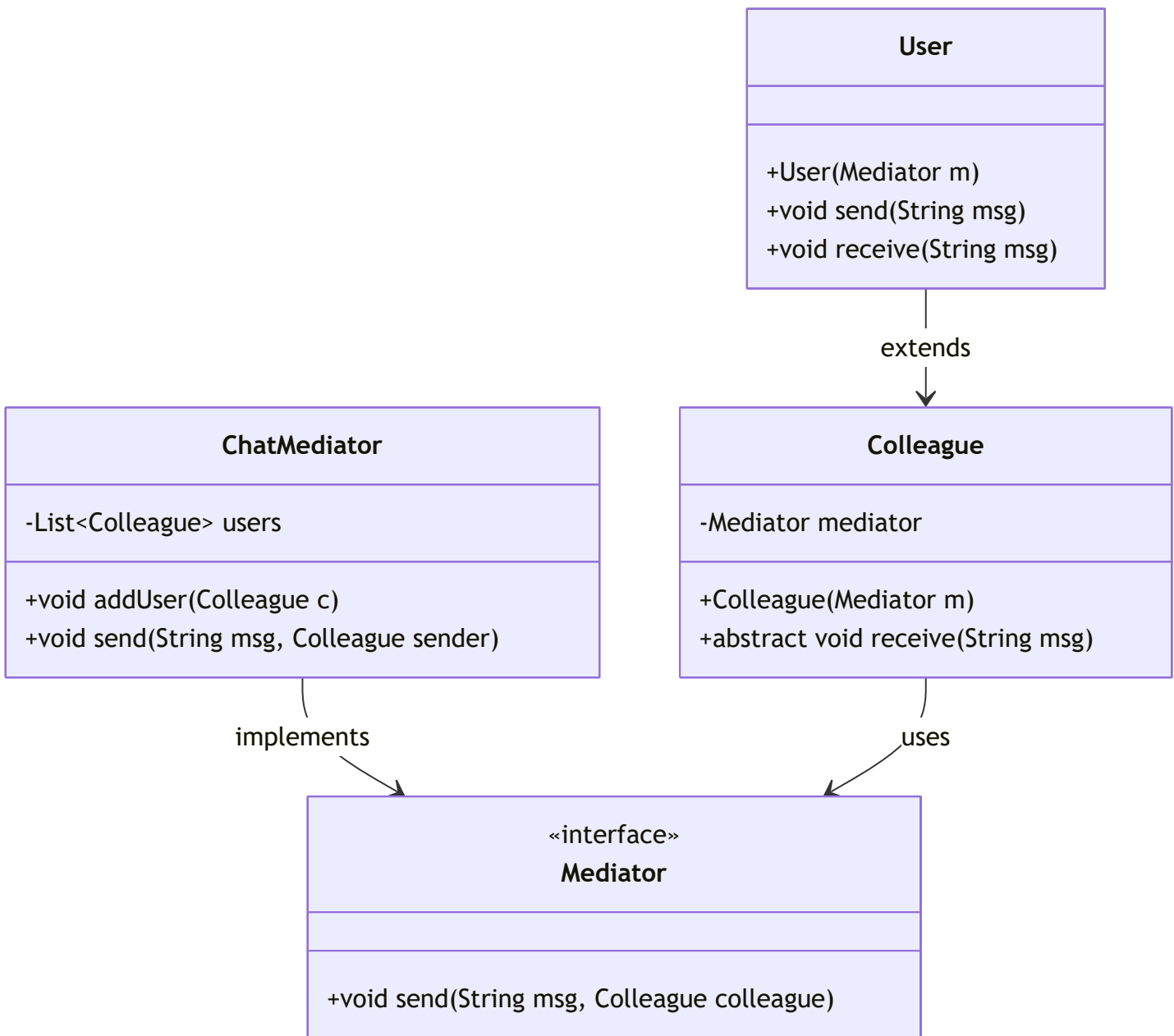
⑦ Mediator 패턴

```
interface Mediator {
    void send(String msg, Colleague colleague);
}

class ChatMediator implements Mediator {
    private List<Colleague> users = new ArrayList<>();
    void addUser(Colleague c) { users.add(c); }
    public void send(String msg, Colleague sender) {
        for (Colleague u : users)
            if (u != sender) u.receive(msg);
    }
}

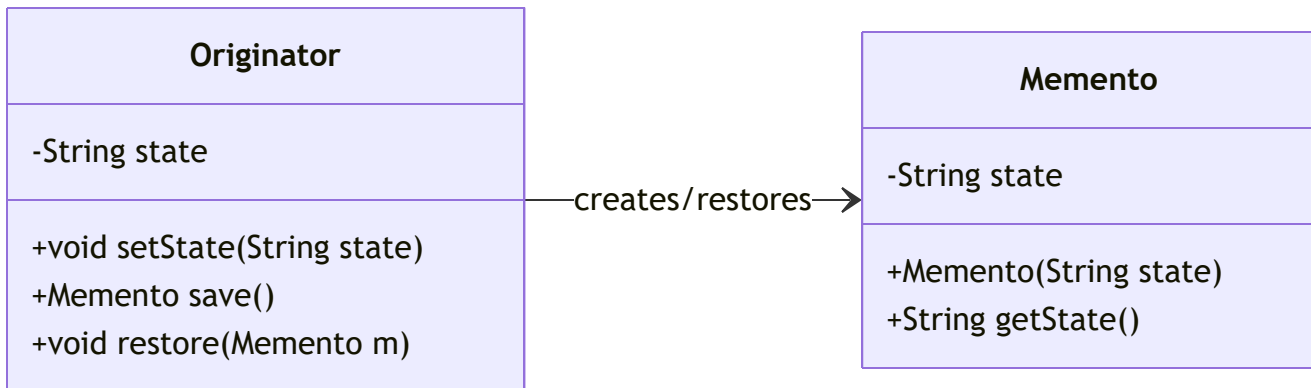
abstract class Colleague {
    protected Mediator mediator;
    Colleague(Mediator m) { this.mediator = m; }
    abstract void receive(String msg);
}

class User extends Colleague {
    User(Mediator m) { super(m); }
    void send(String msg) { mediator.send(msg, this); }
    void receive(String msg) { System.out.println("메시지 수신: " + msg); }
}
```



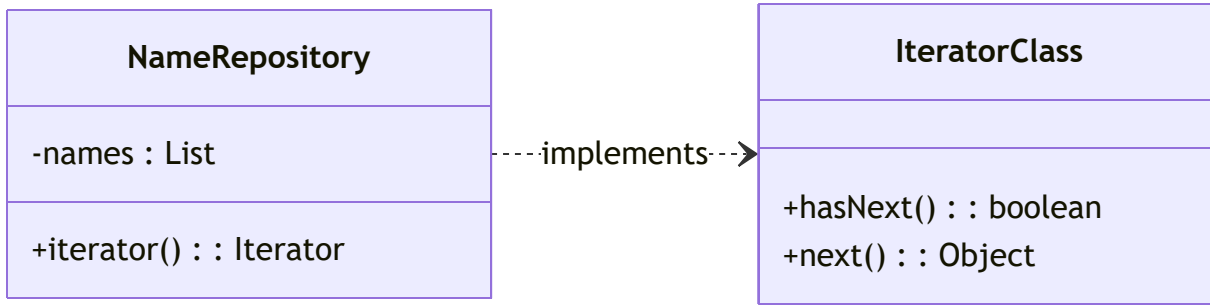
⑧ Memento 패턴

```
class Memento {  
    private String state;  
    Memento(String state) { this.state = state; }  
    String getState() { return state; }  
}  
  
class Originator {  
    private String state;  
    void setState(String state) { this.state = state; }  
    Memento save() { return new Memento(state); }  
    void restore(Memento m) { this.state = m.getState(); }  
}
```



⑨ Iterator 패턴

```
import java.util.*;  
  
class NameRepository implements Iterable<String> {  
    private List<String> names = Arrays.asList("Tom", "Jane", "John");  
    public Iterator<String> iterator() {  
        return names.iterator();  
    }  
}
```



⑩ Visitor 패턴

```

interface Visitor {
    void visit(Book b);
}

class Book {
    void accept(Visitor v) { v.visit(this); }
}

class PriceVisitor implements Visitor {
    public void visit(Book b) { System.out.println("가격 계산 중..."); }
}
  
```

